ZedBoard FMC Digital I/O Bring-Up

For NEMO Chip Test Bench Verification

September 14, 2026

1 Objective

The lab's ZedBoard is connected via its FMC (FPGA Mezzanine Card) connector to a custom test setup used to characterize NEMO ASIC test chips. Before any signal from the FPGA is trusted to drive or read the actual chip, the goal was to build and verify a generic, reusable set of digital input/output test infrastructure on the FPGA side, and to prove — using the FPGA itself, with no external test equipment required — that every FMC pin in the chain (FPGA package pin → FMC connector → breakout board → physical test point) is electrically sound.

This document is a step-by-step record of that process: what was checked, what code was written, what physical hardware issues were discovered along the way, and how the final self-check was performed and verified.

2 Hardware Overview

- **Carrier board:** Digilent ZedBoard (Xilinx/AMD Zynq-7000 SoC, part `XC7Z020-CLG484-1`).
- **Connection to PC:** USB, enumerating as an FTDI FT232-based USB-serial converter (VID 0403 / PID 6014) for UART, separate from the JTAG programming path used by Vivado's Hardware Manager.
- **FMC connector on the ZedBoard:** labeled `FMC-LPC` (Low Pin Count variant of the VITA-57.1 FMC standard).
- **Daughter board:** a Hi-Tech Global `HTG-FMC-SMA-LVDS REV.1.0` board, plugged into the ZedBoard's FMC-LPC socket. This board breaks out:
  - 8 high-speed serial transceiver channels plus 2 reference clock pairs, onto 38 SMA coaxial connectors (labeled `X1`–`X38`, with signal names such as `M2C_0P/N`, `C2M_0P/N`, `CK0/CK1_M2C`, `CLK_P/N`).
  - The FMC's 34 `LA` (Low-pin-count Address, i.e. general-purpose parallel I/O) differential pairs, onto two standard 0.1″ pin headers, `J2` and `J3`.
- **Vivado:** two installations were found on the lab PC, `2015.2` and `2022.1` (both full design editions, not Lab Edition). `2022.1` was used, since it includes the ZedBoard board-file preset and modern SystemVerilog/debug support.

3 Repository / File Overview

All files below live in the project folder `C:\Users\LAB\Desktop\FPGA_PROJECT_YR` (originally named with an `&` character, renamed because Vivado does not accept `&` in project paths).

File	Purpose
`PINS_layout.xdc`	Original lab constraints file: 32 FMC `LA_P` pins as outputs only, 5 switches, 1 clock. Starting point for this work.
`pin_mapping (FMC-FPGA).xlsx`	Lab-authored spreadsheet: FMC connector pad → FMC signal name → FPGA package pin → XDC port name, for all 32 `LA_P` pins plus clock/switches.
`fpga_package_pin_to_fmc.pdf`	Digilent's official ZedBoard FMC pinout chart. Used to independently verify the spreadsheet above (they matched exactly).
`single_pin_tester.v`	Pre-existing lab file: drives one of 32 output pins, selected by a 5-bit switch value, with a 1 MHz square wave. Reference/legacy design.
`generic_fmc_pin.sv`	New: one parameterized bidirectional FMC pin cell (static level, PWM, or input-capture), written for future full NEMO I/O control.
`generic_fmc_io_top.sv / .xdc`	New: instantiates `generic_fmc_pin` 32×, fully configurable per pin via source-code tables (direction, static value, PWM duty). Not yet used on hardware; intended for later, full NEMO testing once bring-up is complete.
`fmc_loopback_test.sv / .xdc`	New: bring-up self-test #1. Pins 0–15 as outputs (automatic walking-one pattern), pins 16–31 as inputs (captured, shown on LEDs). Built and programmed onto the board first; superseded by the design below for more controllable testing.
`fmc_selectable_loopback.sv / .xdc`	New: bring-up self-test #2, **the design actually used for verification**. Switch-selectable pin pair, manual drive value, on-board PASS/FAIL indicator. Described in detail in Section 9.
`fmc_clock_loopback.sv / .xdc`	New: bring-up self-test #3, **used for dynamic (toggling clock) pin verification**. Same switch-selectable pair convention, but drives a real 1 MHz square wave instead of a static level, with a debounced, noise-tolerant sticky fault latch. Described in detail in Section 14.
`fmc_clock_loopback_slow.sv`	New: identical design to `fmc_clock_loopback.sv`, but with a ∼100 Hz clock instead of 1 MHz, specifically so a handheld multimeter can accurately measure it (a standard multimeter cannot correctly average a 1 MHz square wave). Reuses `fmc_clock_loopback.xdc` unchanged, since the port list is identical. **This is the file used for the multimeter measurement in Section 18.**

4 Step 1: Verifying the ZedBoard Is Connected

Before any FPGA work, the ZedBoard's presence on the PC was confirmed at the OS level: Windows enumerates an FTDI FT232-based *USB Serial Converter* (VID_0403&PID_6014). No virtual COM port had been created for it (only the base `FTDIBUS` driver was bound), which was noted but not required for JTAG-based programming and was left unresolved since it did not block the rest of the work.

5 Step 2: Understanding the Existing Pin Mapping

The lab's existing files (`PINS_layout.xdc`, `pin_mapping (FMC-FPGA).xlsx`, `single_pin_tester.v`) were read and cross-checked against each other and against Digilent's official `fpga_package_pin_to_fmc.pdf`. All three sources agreed exactly on which FPGA package pin (e.g. `M19`) corresponds to which FMC connector pad (e.g. `G6`) and which `LA` signal name (e.g. `LA00_P_CC`), for all 32 `LA_P` pins used. This gave a verified foundation for every constraints file written afterward.

6 Step 3: Generic Bidirectional I/O Module (For Future NEMO Control)

Before any physical hardware was touched, a general-purpose, fully source-configurable 32-pin I/O module was written, intended for the eventual real NEMO signal testing (not used on hardware yet at the time of writing):

- `generic_fmc_pin.sv`: one bidirectional pin. As an output it drives either a static level or a PWM waveform (a pseudo-analog stimulus, intended to be smoothed by an external RC filter since the FMC `LA` pins have no true DAC/ADC behind them). As an input it double-flop synchronizes and captures the pad level.
- `generic_fmc_io_top.sv`: instantiates the cell above 32 times over a single `fmc_pins[31:0]` bus. Every pin's direction, drive mode, static value, and PWM duty cycle is set by editing four small look-up tables at the top of the file — no runtime protocol, no software, just re-synthesize after editing.
- `generic_fmc_io_top.xdc`: same 32 physical pins as `PINS_layout.xdc`, renamed to the bidirectional `fmc_pins` port, `LVCMOS18` (the FMC's `VADJ` level on this board).

7 Step 4: First Bring-Up Test — Automatic Walking Pattern

To get something running on real hardware quickly, a simpler design was built first:

Listing 1: Core logic of fmc_loopback_test.sv

```
// Pins 0-15 = outputs, walking one bit every ~0.5s
reg [3:0] idx;
always @(posedge clk) if (tick) idx <= idx + 1;
wire [15:0] out_pattern = 16'b1 << idx;
assign fmc_pins[15:0] = out_pattern;

// Pins 16-31 = inputs, synchronized and captured
always @(posedge clk) begin
```

```
    sync0      <= fmc_pins[31:16];
    in_capture <= sync0;
end
assign led = page_sel ? in_capture[15:8] : in_capture[7:0];
```

This design cycles a single "1" bit through the 16 output pins automatically, and mirrors the 16 input pins onto the board's 8 LEDs (paged by switch 0). It was built, synthesized, implemented, and programmed onto the board successfully in Vivado, and confirmed the toolchain and basic pin constraints were correct. It was then superseded by the design in the next section, which gives manual, deterministic control instead of an automatic timed pattern — much easier to correlate cause and effect on a single pin pair.

8 Step 5: Building and Programming in Vivado 2022.1

1. Project created: `File` → `New Project`, RTL project, part `xc7z020clg484-1` selected directly (the ZedBoard board-file did not successfully register with Vivado's board browser, so the part was selected manually instead — electrically identical outcome for this pure-RTL, no-block-design flow).
2. Sources and constraints added via `Add Sources`.
3. `Flow Navigator` → `Run Synthesis` → `Run Implementation` → `Generate Bitstream`.
4. `Open Hardware Manager` → `Program Device`, bitstream programmed over JTAG. Confirmed successful via the Tcl console log (`End of startup status: HIGH`) and the board's blue `DONE` LED.

9 Step 6: Selectable Self-Check Design (Used for Verification)

The final design used for actual pin-by-pin verification, `fmc_selectable_loopback.sv`, gives full manual control and an on-board PASS/FAIL result, with no oscilloscope or external logic analyzer required:

Listing 2: Core logic of fmc_selectable_loopback.sv

```
wire [3:0] pair_sel  = sw[3:0];   // which of the 16 pairs (0-15) to test
wire       drive_val = sw[4];     // the value to drive on that pair's
   output pin

// Drive only the selected output pin; every other output held low
wire [15:0] out_sel = 16'b1 << pair_sel;
generate
    for (g = 0; g < 16; g = g + 1) begin : g_out
        assign fmc_pins[g] = out_sel[g] ? drive_val : 1'b0;
    end
endgenerate

// Synchronize + capture all 16 input pins
always @(posedge clk) begin
    sync0      <= fmc_pins[31:16];
    in_capture <= sync0;
end
```

```
wire read_back = in_capture[pair_sel];

assign led[0] = drive_val;                // LD0: value being driven right
    now
assign led[1] = read_back;                 // LD1: value read back on the
    paired input
assign led[2] = (drive_val == read_back); // LD2: PASS indicator
```

Pin numbering convention used throughout this design: pin index i for $i \in [0, 15]$ is always an *output* pin (FMC signal `LA`i`_P`); pin index $i + 16$ is always the corresponding *input* pin (FMC signal `LA`$(i{+}16)$`_P`). Switch bits `sw[3:0]` select i; switch bit `sw[4]` sets the value driven on the output side.

10 Step 7: Discovering the SMA Jacks Were the Wrong Physical Connector

Photographs of the daughter board's SMA connectors showed silkscreen labels such as `M2C_0P/N`, `C2M_0P/N`, `CK0/CK1_M2C_P/N`, `CLK_P/N`. A web search of the board's own silkscreen text (`HTG-FMC-SMA-LVDS REV.1.0`, Hi-Tech Global) found the manufacturer's product page, which states the board provides:

> "access to **8 Serial Transceivers** of a host FPGA through **32 SMA connectors** (18GHz)... and **34 pairs of differential** (or 68 single-ended) signals through **standard pin headers**."

This confirmed the SMA jacks are wired to the FMC's high-speed *serial transceiver* pins, not the parallel `LA` GPIO bus that `fmc_selectable_loopback.sv` drives and reads — the two are electrically and functionally unrelated connector groups on the same daughter board.

A further check was made into whether this even mattered practically: the exact FPGA part on the ZedBoard, `XC7Z020` (any package), has **zero GTX transceivers** — confirmed against Xilinx/AMD's own Zynq-7000 Product Selection Guide. This also explained why `fpga_package_pin_to_fmc.pdf` listed the transceiver-related FMC pins (`DP0-C2M_P/N`, `DP0-M2C_P/N`) as *Not Connected* on this board — there is no transceiver hardware on this chip for Digilent to have wired them to in the first place. This ruled out any transceiver-based use of the SMA connectors on this particular hardware combination, regardless of what FPGA logic might be written.

Conclusion: the correct physical connection point for testing the `LA` pins is the daughter board's standard pin headers, `J2` and `J3` — not the SMA jacks.

11 Step 8: Mapping the LA Pins to Physical Header Positions

Through direct inspection/verification by the lab (cross-referenced against `pin_mapping (FMC-FPGA).xlsx`), the following convention was established for the `HTG-FMC-SMA-LVDS` board's headers:

LA index range	Physical header	Notes
0 – 16	`J3`	position number on header = LA index number
17 – 33	`J2`	position number on header = LA index number

Each header position has two columns, `N` and `P`; since the design only drives/reads the single-ended `_P` signal for each `LA` pair, only the `P` column is used.

Combining this with the design's fixed pair convention (output i, input i+16, for $i = 0 \ldots 15$) gives the full physical test table:

Pair (`sw[3:0]`)	Output pin	Input pin
0	J3, pos. 0	J3, pos. 16
1	J3, pos. 1	J2, pos. 17
2	J3, pos. 2	J2, pos. 18
3	J3, pos. 3	J2, pos. 19
4	J3, pos. 4	J2, pos. 20
5	J3, pos. 5	J2, pos. 21
6	J3, pos. 6	J2, pos. 22
7	J3, pos. 7	J2, pos. 23
8	J3, pos. 8	J2, pos. 24
9	J3, pos. 9	J2, pos. 25
10	J3, pos. 10	J2, pos. 26
11	J3, pos. 11	J2, pos. 27
12	J3, pos. 12	J2, pos. 28
13	J3, pos. 13	J2, pos. 29
14	J3, pos. 14	J2, pos. 30
15	J3, pos. 15	J2, pos. 31

12 Step 9: Physical Verification of Pair 0

Using a female-to-female jumper wire (not an SMA cable — the header pins are standard 0.1″ spaced pins, electrically and mechanically unrelated to the SMA jacks discussed in Step 7), header `J3` position 0 (P column, output pin 0) was connected to header `J3` position 16 (P column, input pin 16), with `sw[3:0]=0000` selecting pair 0.

Condition	LD0 (driven)	LD1 (read back)	LD2 (PASS)
Before wiring, `sw[4]=0`	0	1 (floating pin)	0
Wire connected, `sw[4]=0`	0	0	1
Wire connected, `sw[4]=1`	1	1	1

Interpretation: before the jumper was connected, input pin 16 was floating (undriven) and happened to read as a stray logic 1, so the driven value (0) and the readback (1) disagreed and PASS was off — expected, since nothing was physically linked yet. Once the jumper was connected, pin 16 stopped floating and instead reported exactly whatever voltage pin 0 was putting out: 0 V when `sw[4]=0`, and 1.8 V when `sw[4]=1`. In both driven states, the readback matched the drive value and PASS stayed lit. This confirms the entire signal path — FPGA output buffer, PCB trace, FMC connector, header pin 0, the jumper wire, header pin 16, FMC connector, PCB trace, and FPGA input buffer/synchronizer — is electrically sound for this pin pair.

13 SMA Connector Usability Survey

Following on from the discovery in Step 7 above, every one of the 38 SMA jacks was individually classified against Digilent's official ZedBoard FMC pinout PDF, to determine exactly which, if any, could be used at all on this specific ZedBoard + `HTG-FMC-SMA-LVDS` combination.

SMA	Label	Status	Reason
X31	CK0_M2C_P	Usable*	FMC_CLK0_P → pkg L18, real trace
X32	CK0_M2C_N	Usable*	FMC_CLK0_N → pkg L19, real trace
X13	CK1_M2C_P	Usable*	FMC_CLK1_P → pkg D18, real trace
X14	CK1_M2C_N	Usable*	FMC_CLK1_N → pkg C19, real trace
X33	M2C_0P	Not usable	DP0 lane, "Not Connected" on ZedBoard (no GTX)
X34	M2C_0N	Not usable	DP0 lane, same
X15	C2M_0P	Not usable	DP0 lane, same
X16	C2M_0N	Not usable	DP0 lane, same
X35	M2C_1P	Not usable	DP1 lane, does not exist on LPC connector
X36	M2C_1N	Not usable	DP1 lane, same
X17	C2M_1P	Not usable	DP1 lane, same
X18	C2M_1N	Not usable	DP1 lane, same
X25	M2C_2P	Not usable	DP2 lane, LPC does not carry it
X26	M2C_2N	Not usable	DP2 lane, same
X7	C2M_2P	Not usable	DP2 lane, same
X8	C2M_2N	Not usable	DP2 lane, same
X27	M2C_3P	Not usable	DP3 lane, LPC does not carry it
X28	M2C_3N	Not usable	DP3 lane, same
X9	C2M_3P	Not usable	DP3 lane, same
X10	C2M_3N	Not usable	DP3 lane, same
X29	M2C_4P	Not usable	DP4 lane, LPC does not carry it
X30	M2C_4N	Not usable	DP4 lane, same
X11	C2M_4P	Not usable	DP4 lane, same
X12	C2M_4N	Not usable	DP4 lane, same
X19	M2C_5P	Not usable	DP5 lane, LPC does not carry it
X20	M2C_5N	Not usable	DP5 lane, same
X1	C2M_5P	Not usable	DP5 lane, same
X2	C2M_5N	Not usable	DP5 lane, same
X21	M2C_6P	Not usable	DP6 lane, LPC does not carry it
X22	M2C_6N	Not usable	DP6 lane, same
X3	C2M_6P	Not usable	DP6 lane, same
X4	C2M_6N	Not usable	DP6 lane, same
X23	M2C_7P	Not usable	DP7 lane, LPC does not carry it
X24	M2C_7N	Not usable	DP7 lane, same
X5	C2M_7P	Not usable	DP7 lane, same
X6	C2M_7N	Not usable	DP7 lane, same
X37	CLK_P	Not usable	Transceiver reference clock input, irrelevant with no GTX
X38	CLK_N	Not usable	Transceiver reference clock input, same

Summary: 4 of 38 SMA jacks are electrically connected (the two clock pairs), and **34 are not usable at all** on this hardware combination. **Even the 4 "usable" jacks are dedicated differential clock inputs, not general-purpose GPIO – using them would require a clock-buffer primitive (IBUFDS), not the plain LVCMOS loopback logic used throughout this document.* For all practical purposes in this project (digital I/O self-check, and eventual NEMO signal testing), **the J2/J3 pin headers are the only usable connectors** on this daughter board.

14 Step 10: Clock-Based Loopback Test — Design

All verification up to this point used a *static* drive value (`sw[4]` held at a constant 0 or 1). This step extends the same self-check idea to a *dynamic* signal — a continuously toggling clock — since a real chip clock or data line needs to carry a changing signal correctly, not just hold a DC level.

`fmc_clock_loopback.sv` reuses the exact same pin/pair convention as `fmc_selectable_loopback.sv` (output pin i, input pin $i+16$, selected by `sw[3:0]`), but `sw[4]` now *enables a 1 MHz, 50% duty square wave* on the output pin instead of setting a static level:

Listing 3: Core clock-generation and comparison logic of fmc_clock_loopback.sv

```
// synchronize the raw mechanical switch into this clock domain
reg sw4_sync0, sw4_sync1;
always @(posedge clk) begin
    sw4_sync0 <= sw[4];
    sw4_sync1 <= sw4_sync0;
end
wire clk_en = sw4_sync1;

// 1 MHz, 50% duty square wave (100 MHz / 100)
reg [6:0] div = 0;
always @(posedge clk)
    div <= (div == 99) ? 0 : div + 1;
wire clk_1mhz  = (div < 50);
wire drive_val = clk_en & clk_1mhz;

// drive only the selected output pin; every other output pin held low
assign fmc_pins[g] = out_sel[g] ? drive_val : 1'b0;  // per bit, g=0..15

// synchronize + capture all 16 input pins (as before)
always @(posedge clk) begin
    sync0      <= fmc_pins[31:16];
    in_capture <= sync0;
end
wire read_back = in_capture[pair_sel];

// delay the drive value by the same 2 cycles the input path takes, so the
// comparison lines up in time instead of mismatching every single cycle
// purely from pipeline latency
reg drive_val_d1, drive_val_d2;
always @(posedge clk) begin
    drive_val_d1 <= drive_val;
    drive_val_d2 <= drive_val_d1;
end
wire match_now = (drive_val_d2 == read_back);
```

Inputs and outputs used, explicitly:

- **Output pin** (drives the clock): `fmc_pins[sw[3:0]]`, i.e. FMC signal `LAsw[3:0]_P`. This is the only pin the FPGA actively places a voltage on.
- **Input pin** (reads the clock back): `fmc_pins[sw[3:0]+16]`, i.e. FMC signal `LA(sw[3:0]+16)_P`. The FPGA only listens on this pin; it never drives it.

- **Control inputs**: `sw[3:0]` selects which of the 16 pairs is under test; `sw[4]` enables/disables the 1 MHz drive.
- **Result outputs**: `LD0` (live driven clock), `LD1` (live read-back clock), `LD2` (live match), `LD3` (sticky fault latch, Section 15).

15 Step 11: Iterating on the Fault Detector

The first version of the fault indicator simply latched permanently on *any* single mismatch between the (delay-compensated) drive value and the readback, for as long as `sw[4]` stayed high. In practice this proved too strict, and was refined twice:

1. **Problem 1 — switch bounce.** `sw[4]` is a mechanical toggle switch. On the first test run, the fault latch tripped every time, even though the live match indicator (`LD2`) looked solidly correct. The likely cause: a mechanical switch's contacts bounce (rapidly make/break) for a few milliseconds when flipped, which at 100 MHz is hundreds of thousands of clock cycles — plenty of opportunity for a spurious transient mismatch right at the moment of the flip, even though the steady-state connection is fine. **Fix:** a proper 2-flip-flop synchronizer was added on `sw[4]`, and the fault latch was changed to ignore all comparisons for a ∼20 ms settle window (2,000,000 cycles) after `sw[4]` goes high, before it starts trusting the comparison at all.
2. **Problem 2 — ordinary electrical noise.** Even after the settle-window fix, the fault latch still tripped occasionally, while `LD2` remained fully, solidly bright (i.e. the vast majority of cycles matched correctly). This pointed to isolated, single-cycle glitches — normal electrical noise expected when running a 1 MHz signal over an unshielded, unterminated jumper wire — rather than a genuinely broken connection, which would show *continuous* disagreement. **Fix:** the fault latch was changed to require 16 *consecutive* mismatched cycles (160 ns, well under one half-period of the 1 MHz signal, but far longer than a one-off noise glitch could plausibly sustain) before declaring a real fault. A single isolated mismatch now resets the run-counter back to zero instead of tripping anything.

After both fixes, re-running the test showed `LD3` staying dark through the full test window, confirming the connection is genuinely sound and the earlier trips were false positives from the detector being too strict, not real hardware faults.

16 Step 12: Physical Verification Results (Pair 0, Clock Test)

Same physical wiring as Step 9 (header `J3` position 0 ↔ position 16, P column), now with the clock design programmed:

Condition	LD0	LD1	LD2	LD3 (sticky fault)
No connection, `sw[4]=0`	0	1 (floating)	0	0
Wire connected, `sw[4]=0`	0	0	1	0
Wire connected, `sw[4]=1`, before fixes	1	1	1	**1 (false trip)**
Wire connected, `sw[4]=1`, after both fixes	1	1	1	**0 (confirmed pass)**

Result: pair 0 is confirmed working both as a static level and as a genuine 1 MHz toggling signal.

17 Verifying a Clock Signal With Certainty

The on-board LEDs (`LD2`, `LD3`) are convenient, but they share one limitation: they all rely on the FPGA's own internal logic judging itself. Section 15 showed concretely why that is not automatically trustworthy — a bug in the checking logic itself (the too-strict original fault latch) produced a false failure that had nothing to do with the physical connection. For a definitive, independent answer, four methods were considered, in increasing order of certainty:

1. **Multimeter, DC voltage mode.** Probing the output pin while `sw[4]=1` should read roughly **$\sim$0.9 V** — the time-average of a 0 V/1.8 V square wave at 50% duty cycle. Reading $\sim$0 V or $\sim$1.8 V instead would indicate the pin is stuck rather than toggling. Fast, but only an indirect, averaged indicator.
2. **Multimeter, frequency/Hz mode (if available).** Some handheld meters can display the measured frequency directly (ideally $\approx$1.00 MHz); many basic meters, however, do not measure reliably above a few hundred kHz, so this is meter-dependent.
3. **Vivado's built-in Integrated Logic Analyzer (ILA).** By marking the key internal signals (`drive_val`, `read_back`, `match_now`) for debug and using `Tools` $\rightarrow$ `Set Up Debug`, a live digital waveform can be streamed back over the existing JTAG link, with no extra equipment. Still, technically, a capture of what the FPGA itself sampled, not a fully independent measurement.
4. **Oscilloscope — the only fully independent method.** Probing the physical output and input pins directly measures the literal voltage on the pin with separate hardware, completely decoupled from the FPGA's own comparison logic. This is the only method that cannot be fooled by a bug in the design's self-check logic (as Section 15 demonstrated can happen). A correct signal should show a clean square wave, period 1 μs (1 MHz), swinging between $\sim$0 V and $\sim$1.8 V, with the duty cycle close to 50% (high time $\approx$ low time $\approx$ 0.5 μs), and clean edges (fast rise/fall, minimal ringing or overshoot). The same check on the paired input pin should show an essentially identical waveform, confirming the signal genuinely propagated through the physical path rather than merely being self-consistent inside the FPGA's own logic.

18 Step 13: Multimeter Verification

As an independent check outside the FPGA's own logic (per the reasoning in Section 17), the clock signal was also measured directly with a handheld multimeter in DC voltage mode.

First attempt — 1 MHz design (`fmc_clock_loopback.sv`) — inconclusive. Probing the output pin (header `J3`, position 0, P column) against a GND reference, with `sw[4]=1`, gave an unstable reading that jumped to $\sim$0.4 V and then settled into jitter between roughly 0.065 V and 0.089 V, rather than the expected $\sim$0.9 V (the time-average of a 0 V/1.8 V, 50% duty square wave). This was determined to be a limitation of the multimeter itself, not a sign of a fault: handheld multimeters are designed for DC and low-frequency signals (their input filtering and ADC integration time is typically only accurate up to a few hundred Hz–kHz) and cannot correctly track or average a signal switching every 500 ns. This did not contradict the digital self-check, which continued to show `LD2` solidly correct throughout.

Second attempt — $\sim$100 Hz design (`fmc_clock_loopback_slow.sv`) — conclusive. A new source file was written, identical to `fmc_clock_loopback.sv` except for a slower clock divider

(∼100 Hz instead of 1 MHz), specifically to bring the signal within the multimeter's usable bandwidth. It reuses `fmc_clock_loopback.xdc` unchanged, as the port list did not change. This was added to the Vivado project, set as top, and rebuilt/reprogrammed through the same flow as every design before it.

Condition	Multimeter (DC volts, pin 0)	LD3 (sticky fault)
`sw[4]=0` (baseline)	∼0 V	—
`sw[4]=1`, pin 16 *not* connected	**0.899 V (stable)**	ON
`sw[4]=1`, pin 16 connected to pin 0	**0.899 V (stable)**	**OFF**

Interpretation: with pin 16 left unconnected, `LD3` correctly reported a fault — not because anything was broken, but because a floating input pin cannot agree with a toggling output pin, so the fault latch was correctly flagging the absence of a completed loop, exactly as designed. Once the jumper wire was connected from pin 0 to pin 16 (in addition to the multimeter probes, which remained on pin 0 the whole time), `LD3` went dark, confirming the loopback path itself is sound. Throughout both conditions, the multimeter reading stayed stable at **0.899 V** — within 0.001 V of the theoretical ideal ($1.8V \times 50\% = 0.9V$) — which by itself, independently of any FPGA-internal logic, confirms the output pin is genuinely toggling at very close to a true 50% duty cycle.

Combined result for pair 0: verified working as a static level (Section 9), as a 1 MHz toggling clock via the internal self-check (Sections 14–15), and now independently confirmed via an external multimeter measurement completely outside the FPGA's own logic. This is the strongest evidence gathered for any single pin pair in this project.

19 Status and Next Steps

- Pair 0 (`LA00_P` ↔ `LA16_P`) is verified working end-to-end: as a static 0/1 level (`fmc_selectable_loopback.sv`), as a genuine 1 MHz toggling clock via the internal self-check (`fmc_clock_loopback.sv`), and independently confirmed with an external multimeter measurement (`fmc_clock_loopback_slow.sv`, 0.899 V, stable) – see Section 18.
- Of the daughter board's 38 SMA jacks, only 4 are electrically connected at all (the two clock pairs), and none of the 38 are usable as general-purpose GPIO test points – the J2/J3 pin headers remain the sole practical connection point (Section 13).
- The remaining 15 pairs (table in Step 8) can be verified the same way, with either design: move the jumper wire per the table, select the pair with `sw[3:0]`, toggle `sw[4]`, and confirm `LD2`/`LD3` (or an oscilloscope, for full certainty) indicate a clean pass.
- Once all pairs are verified (or a sufficient subset to trust the design and wiring), the next step is to move from this self-loopback test to actually connecting and driving/reading the real NEMO chip signals, most likely by returning to the fully configurable `generic_fmc_io_top.sv` design from Step 3 (direction, static/PWM drive, and capture individually configurable per pin) rather than the switch-selected single-pair-at-a-time design used for bring-up.